

A Survey of AI Techniques for LLM Optimization

Presented by: Gabriel, Tom, Youssef, Ali, Maxime



Problem Statement

Area of focus: *Improving training efficiency. Optimizing hyperparameters, learning rates, or data-loading strategies.*

Key number: Gemini 3 has **21 538,35 billion** parameters

Bad hyperparameters can multiply compute costs 2 to 10 times (turning \$100k jobs into \$1M+), and add weeks to months of research delay.



Organization

BOHB: Robust and Efficient Hyperparameter Optimization at Scale

Stefan Falkner¹ Aaron Klein¹ Frank Hutter¹

Abstract

Modern deep learning methods are very sensitive to many hyperparameters, and, due to the long training times of state-of-the-art models, vanilla Bayesian hyperparameter optimization is typically computationally infeasible. On the other hand, bandit-based configuration evaluation approaches based on random search lack guidance and do not converge to the best configurations as quickly. Here, we propose to combine the benefits of both Bayesian optimization and bandit-based methods, in order to achieve the best of both worlds: strong anytime performance and fast convergence to optimal configurations. We propose a new practical state-of-the-art hyperparameter optimization method, which consistently outperforms both Bayesian optimization and Hyperband on a wide range of problem types, including high-dimensional toy functions, support vector machines, feed-forward neural networks, Bayesian neural networks, deep reinforcement learning, and convolutional neural networks. Our method is robust and versatile, while at the same time being conceptually simple and easy to imple-

for a practical solution to the hyperparameter optimization (HPO) problem that fulfills many desiderata:

- 1. Strong Anytime Performance.** Since large contemporary neural networks often require days or even weeks to train, HPO methods that view performance as a black box function to be optimized require extreme resources. The overall budget that most researchers and practitioners can afford during development is often not much larger than that of fully training a handful of models, and hence practical HPO methods must go beyond this blackbox view to already yield good configurations with such a small budget.
- 2. Strong Final Performance.** On the other hand, what matters most at deployment time is the performance of the best configuration a HPO method can find given a larger budget. Since finding the best configurations in a large space requires guidance, this is where methods based on random search struggle.
- 3. Effective Use of Parallel Resources.** With the rise of parallel computing, large parallel resources are often available (e.g., compute clusters or cloud computing), and practical HPO methods need to be able to use these effectively.
- 4. Scalability.** Modern deep neural networks require the

Published as a conference paper at ICLR 2018

LEARNING TO TEACH

Yang Fan¹; Fei Tian^{1*}, Tao Qin¹, Xiang-Yang Li¹ & Tie-Yan Liu¹

¹School of Computer Science and Technology, University of Science and Technology of China
fyabc@mail.ustc.edu.cn, xiangyangli@ustc.edu.cn
²Microsoft Research
{fetia,taoqin,tyliu}@microsoft.com

ABSTRACT

Teaching plays a very important role in our society, by spreading human knowledge and educating our next generations. A good teacher will select appropriate teaching materials, impact suitable methodologies, and set up targeted examinations, according to the learning behaviors of the students. In the field of artificial intelligence, however, one has not fully explored the role of teaching, and pays most attention to machine *learning*. In this paper, we argue that equal attention, if not more, should be paid to teaching, and furthermore, an optimization framework (instead of heuristics) should be used to obtain good teaching strategies. We call this approach “learning to teach”. In the approach, two intelligent agents interact with each other: a student model (which corresponds to the learner in traditional machine learning algorithms), and a teacher model (which determines the appropriate data, loss function, and hypothesis space to facilitate the training of the student model). The teacher model leverages the feedback from the student model to optimize its own teaching strategies by means of reinforcement learning, so as to achieve teacher-student co-evolution. To demonstrate the practical value of our proposed approach, we take the training of deep neural networks (DNN) as an example, and show that by using the learning to teach techniques, we are able to use much less training data and fewer iterations to achieve almost the same accuracy

StarCraft II: A New Challenge for Reinforcement Learning

Oriol Vinyals Timo Ewalds Sergey Bartunov Petko Georgiev
Alexander Sasha Vezhnevets Michelle Yeo Alireza Makhzani Heinrich Küttler
John Agapiou Julian Schrittwieser John Quan Stephen Gaffney Stig Petersen
Karen Simonyan Tom Schaul Hado van Hasselt David Silver Timothy Lillicrap
DeepMind

Kevin Calderone Paul Keet Anthony Brunasso David Lawrence
Anders Ekermo Jacob Repp Rodney Tsing
Blizzard

Abstract

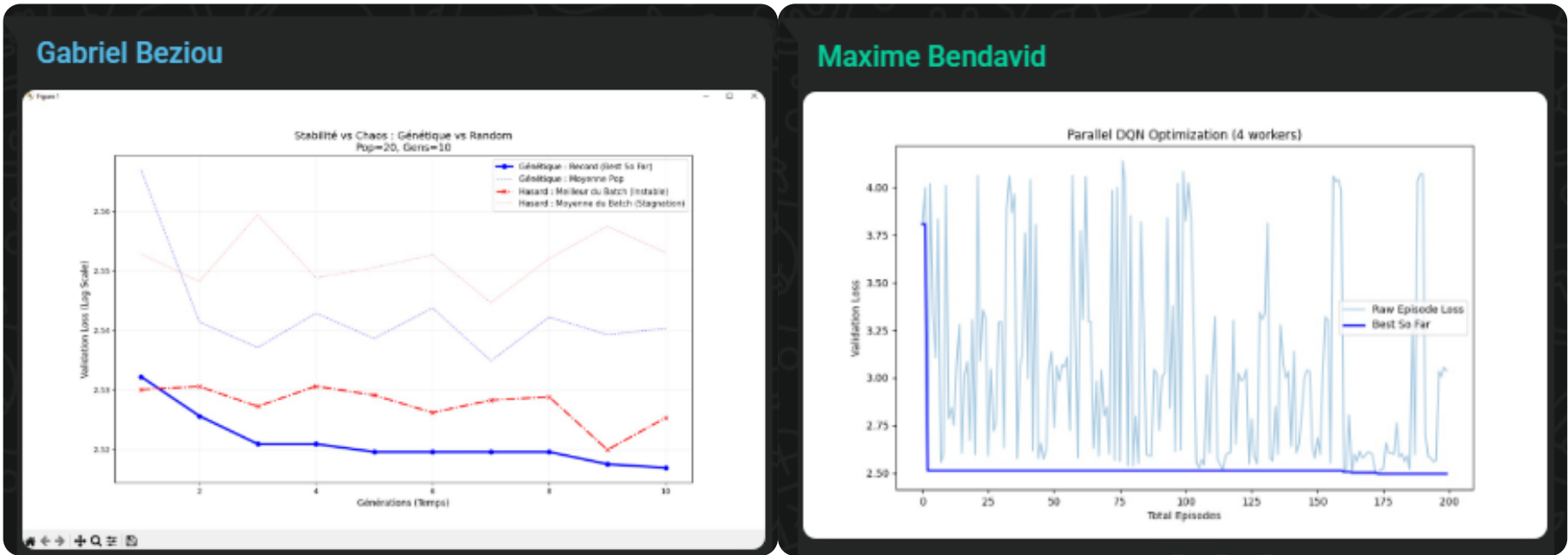
This paper introduces *SC2LE* (StarCraft II Learning Environment), a reinforcement learning environment based on the game StarCraft II. This domain poses a new grand challenge for reinforcement learning, representing a more difficult class of problems than considered in most prior work. It is a multi-agent problem with multiple players interacting; there is imperfect information due to a partially observed map; it has a large action space involving the selection and control of hundreds of units; it has a large state space that must be observed solely from raw input feature planes; and it has delayed credit assignment requiring long-term strategies over thousands of steps. We describe the observation, action, and reward specification for the StarCraft II domain and provide an open source Python-based interface for communicating with the game engine. In addition to the main game maps, we provide a suite of mini-games focusing on different elements of StarCraft II gameplay. For the main game maps, we also provide an accompanying dataset of game replay data from human expert players. We give initial baseline results for neural networks trained from this data to predict game outcomes and

State of Art

- Ali
- Youssef
- Tom

Implementation

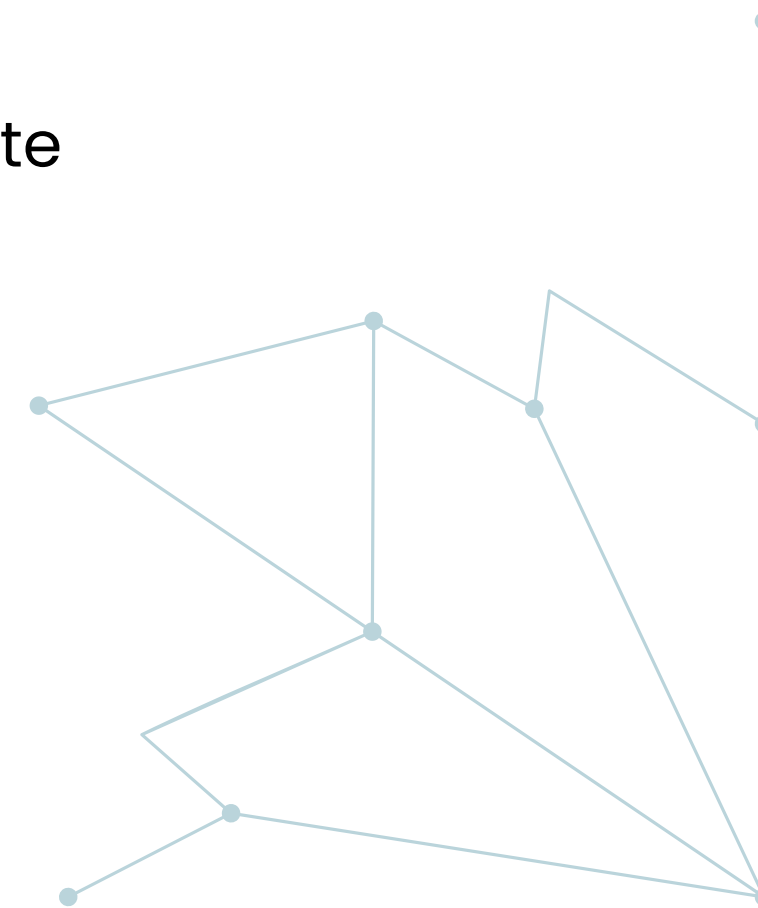
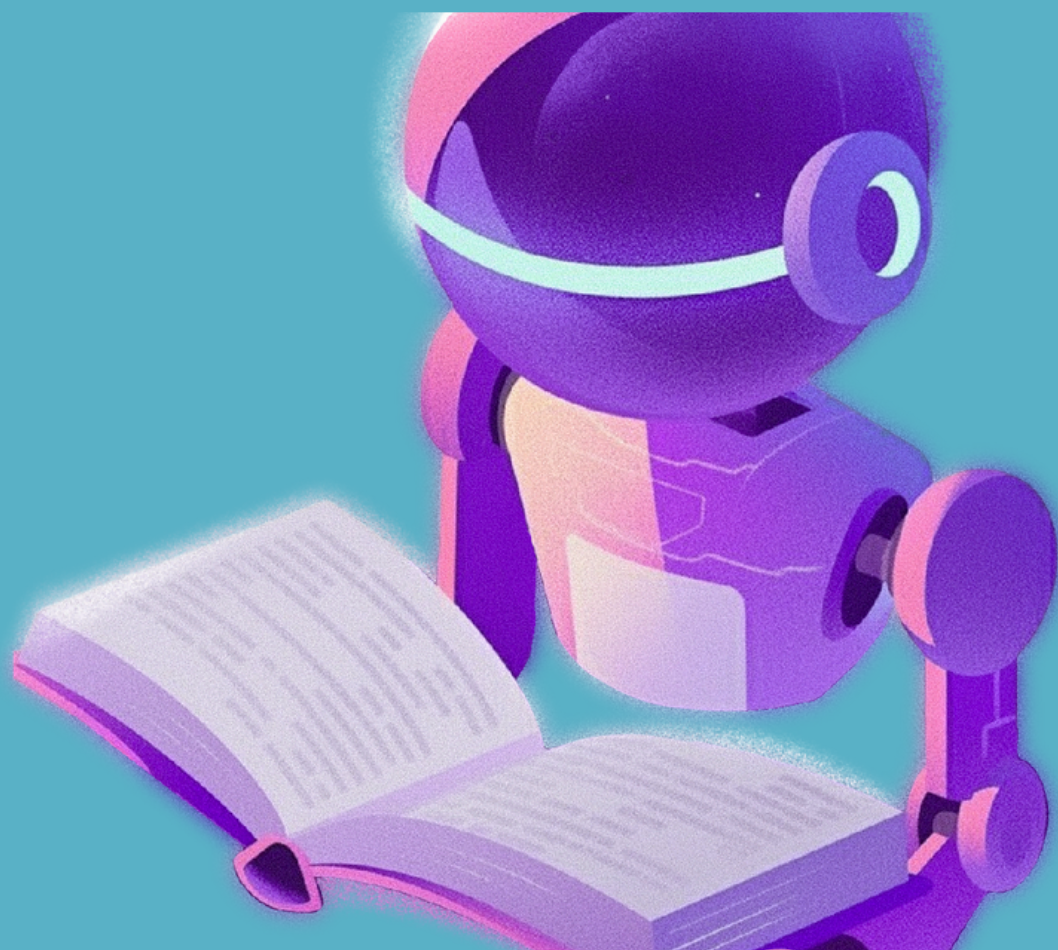
- Maxime
- Gabriel

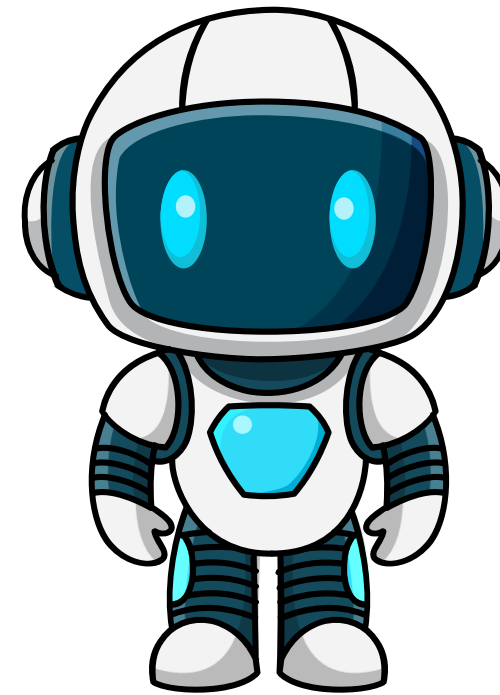




Methods surveyed

- **Data efficiency** : Learning to Teach
- **Hyperparameters optimization** : PBT
- **Training stability**: Learned Learning Rate Schedules





Methods surveyed

- **Scalable hyperparameter search**

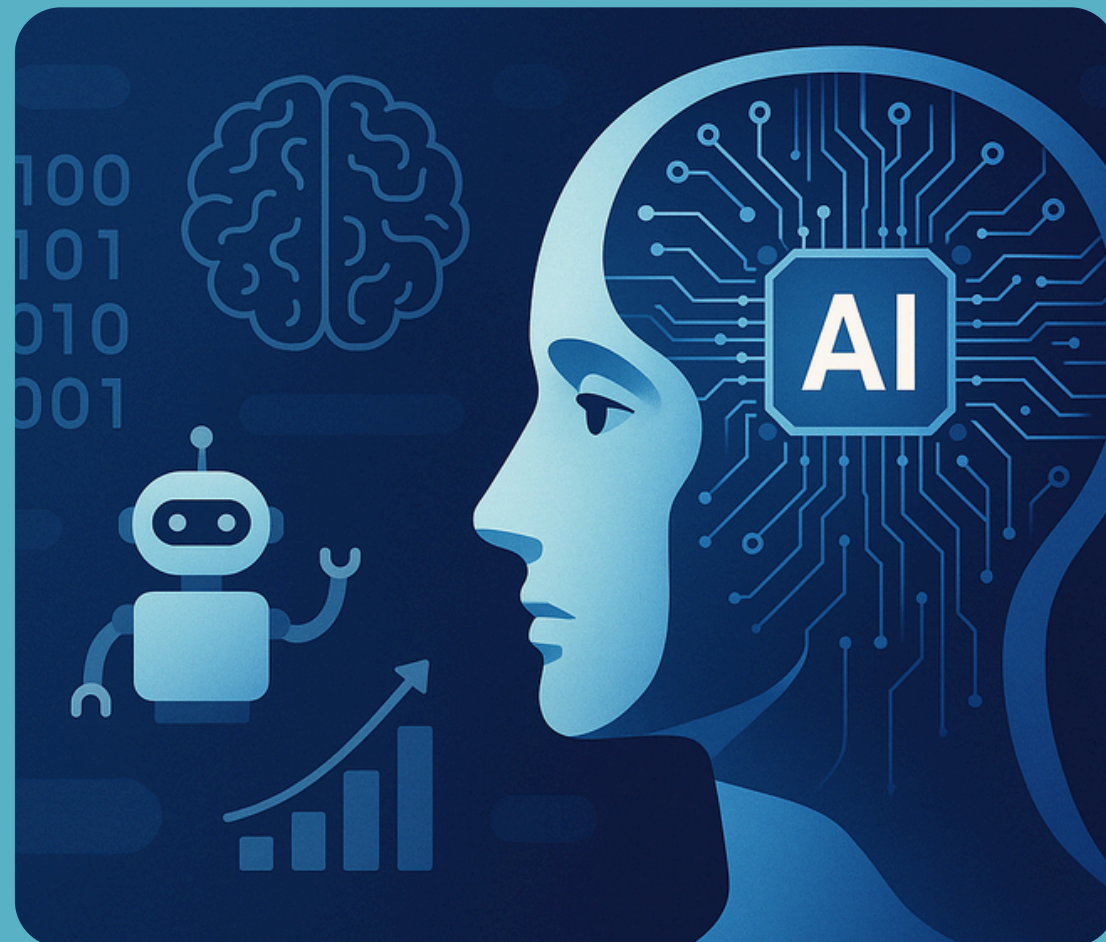
Mixes Hyperband + Bayesian Optimization -> finds better hyperparameters faster than BO, Hyperband, or random search

- **LLM instruction-tuning HPO**

NOMAD and TPE tune LoRA hyperparams better than defaults: improved performance on MMLU, BBH, HumanEval

- **LLM-aided LoRA hyperparameter search**

Turns hyperparams into text, uses LLM embeddings for BO: outperforms standard HPO with big gains on some Benchmarks





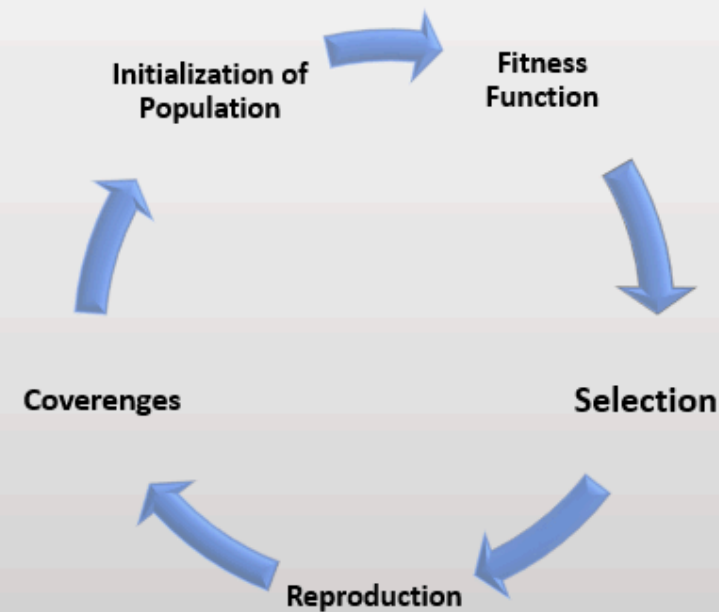
Experimental setup

- NanoGPT
- TinyShakespeare Dataset
- Genetic Algorithm vs Reinforcement Learning for HPO
- 10 gens of 20 individuals $\Leftrightarrow$ 200 Runs of RL



```
# Define the values that hyperparameters can take
self.bounds = {
    'lr': (0.0001, 0.01), # learning rate aka the speed of learning
    'dropout': (0.0, 0.5) # dropout rate aka to prevent the model from overfitting (learning by heart)
}
self.choices = {
    'batch_size': [16, 32, 64, 128], # batch size: number of data samples processed simultaneously
    'n_embd': [32, 64, 128], # (Width): The model's "memorization capacity"
    'n_layer': [2, 4, 6], # Model depth: The "reasoning capacity" (abstraction)
}
```

What is Genetic Algorithm?



www.educb



Genetic Algorithm

1. Genome Representation & Initialization

2. Definition of fitness Function

- $\text{Fitness} = (1 / \text{Validation Loss})$ e.g.

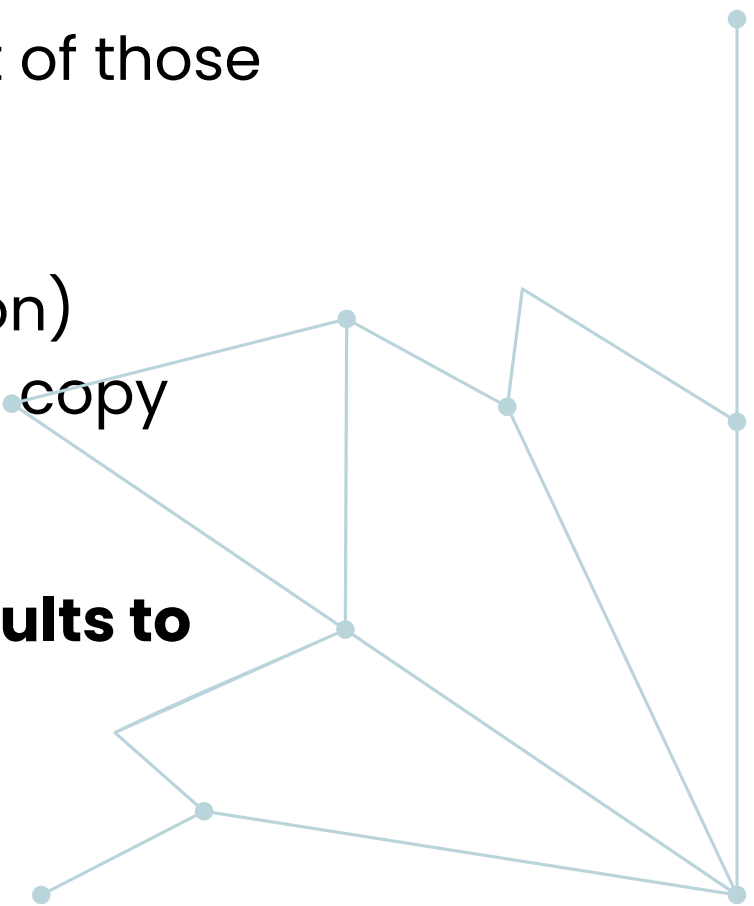
3. Selection Strategy (Tournament Selection)

- 3 random individuals two times, the best of those two rounds are the parents

4. Reproduction (Elitism, Crossover & Mutation)

- We selected the two best individuals and copy them in the next generation

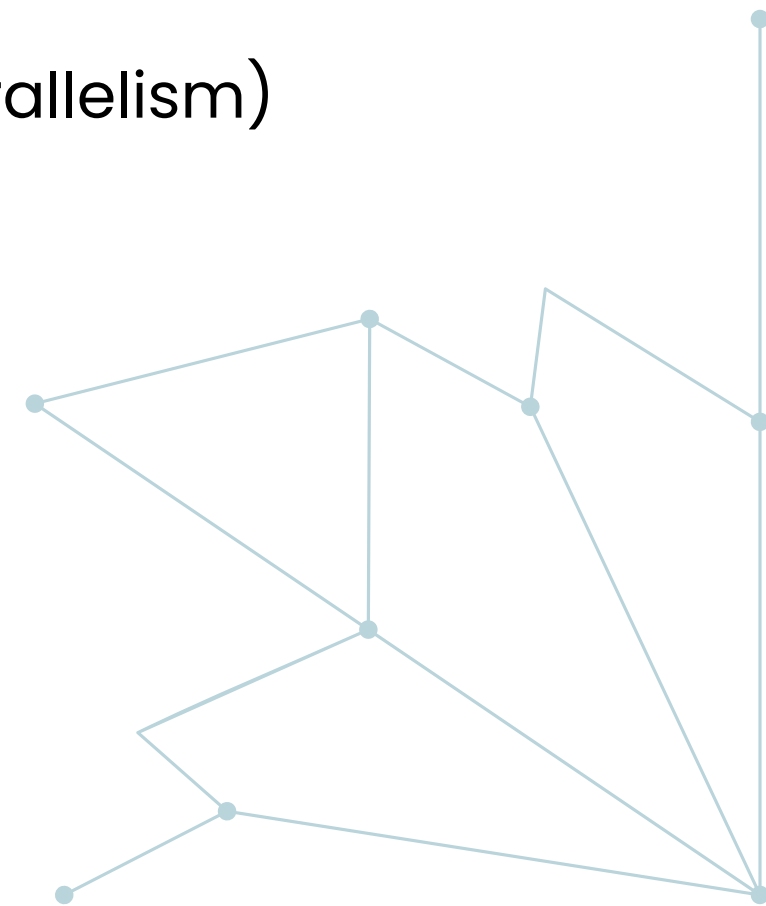
→ **We compared our results VS random results to see if the algorithm is efficient.**



Reinforcement Learning

1. Create the RL Environment
 - a. Markov Decision Process (MDP)
 - b. Reward
2. Q-learning Algorithm
 - a. Q-table
 - b. Epsilon Greedy policy
 - c. Update the rule
3. Training loop
 - a. Reset environment
 - b. Selection loop
 - c. Train & Evaluate
 - d. Backpropagation reward
 - e. Decay epsilon

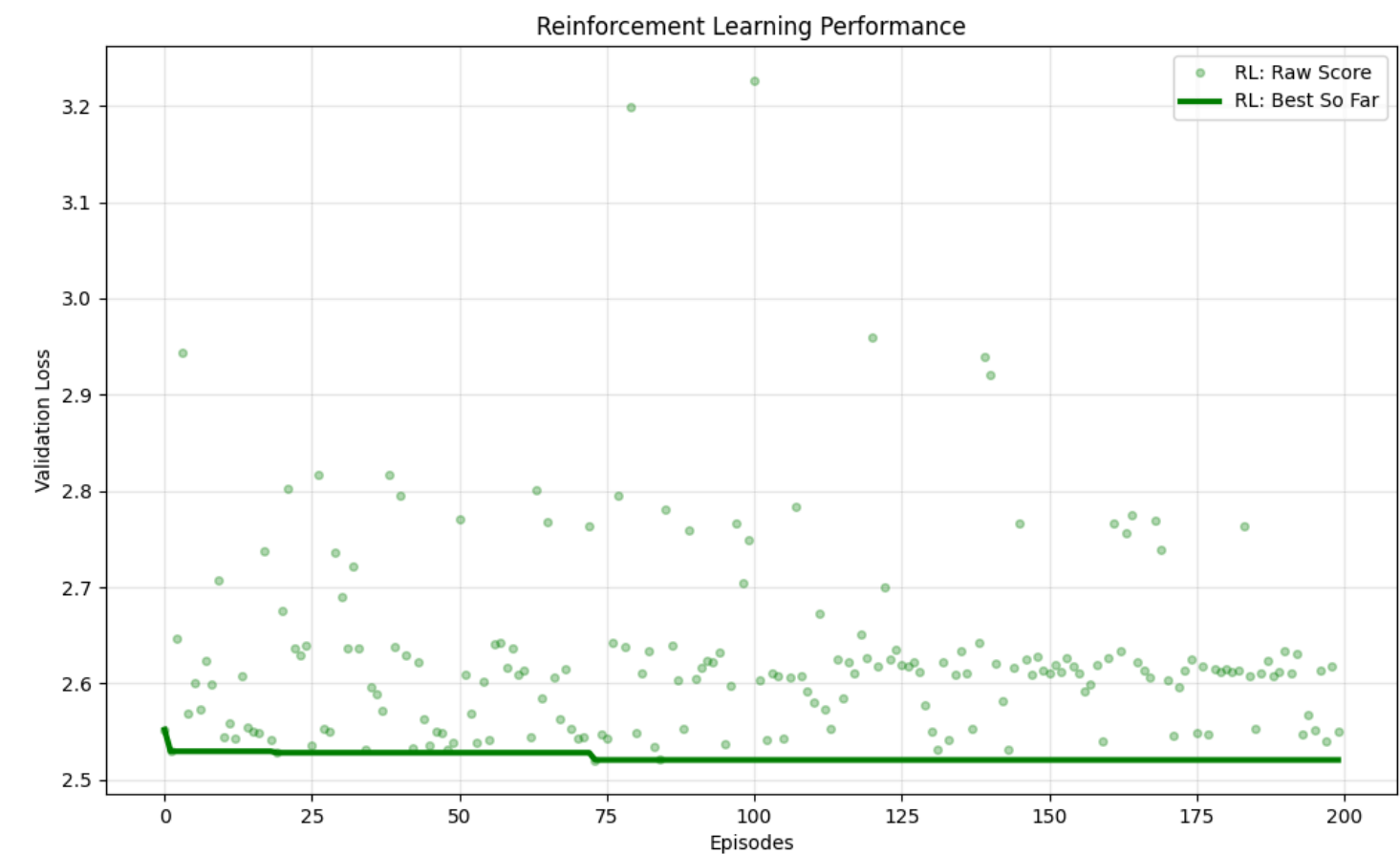
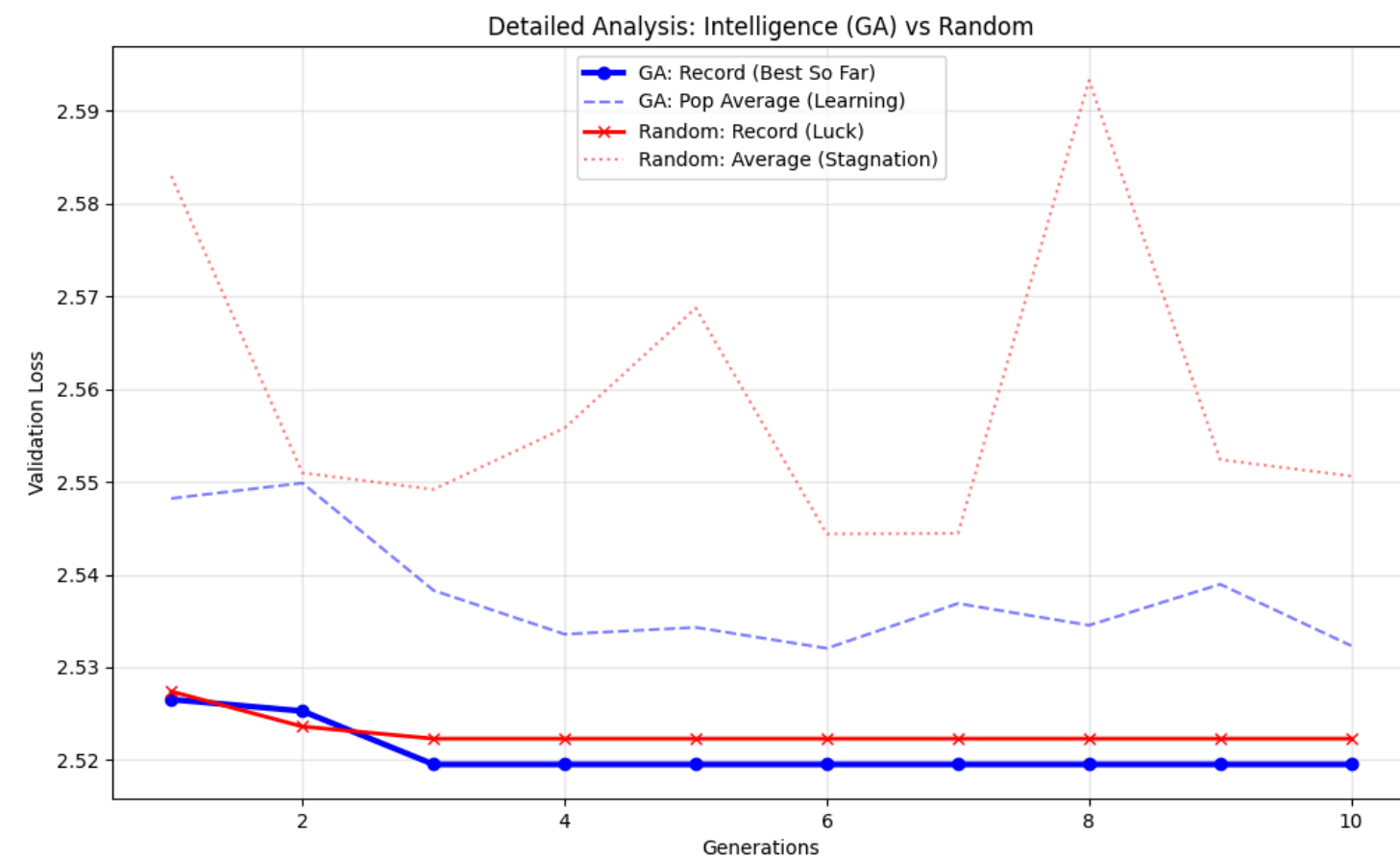
- Q-learning (based on the TD)
- Reward Shaping (Efficiency-aware)
- Deep Q-Learning (DQN)
- Parallelization (Batch Parallelism)
- Final choice



Results

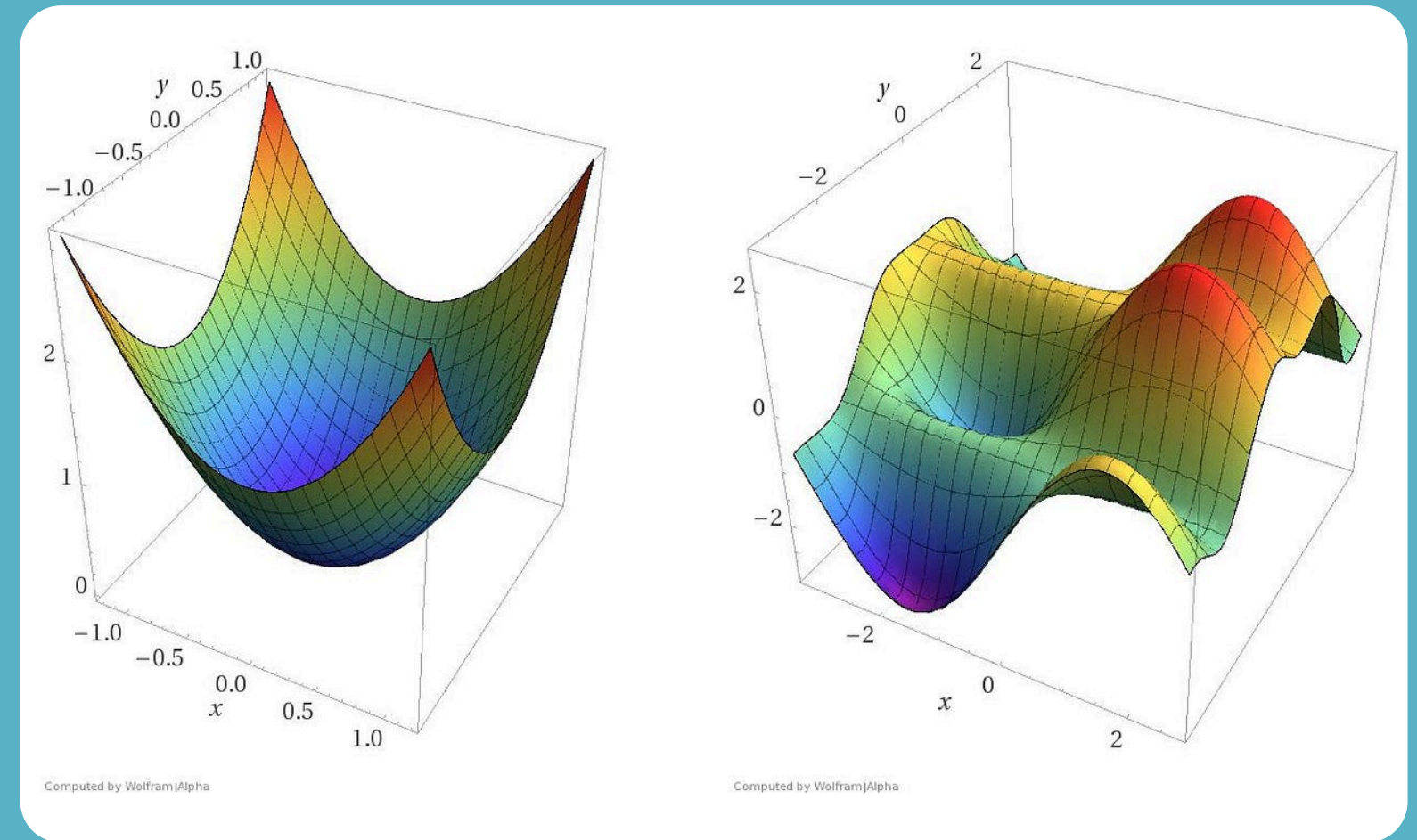
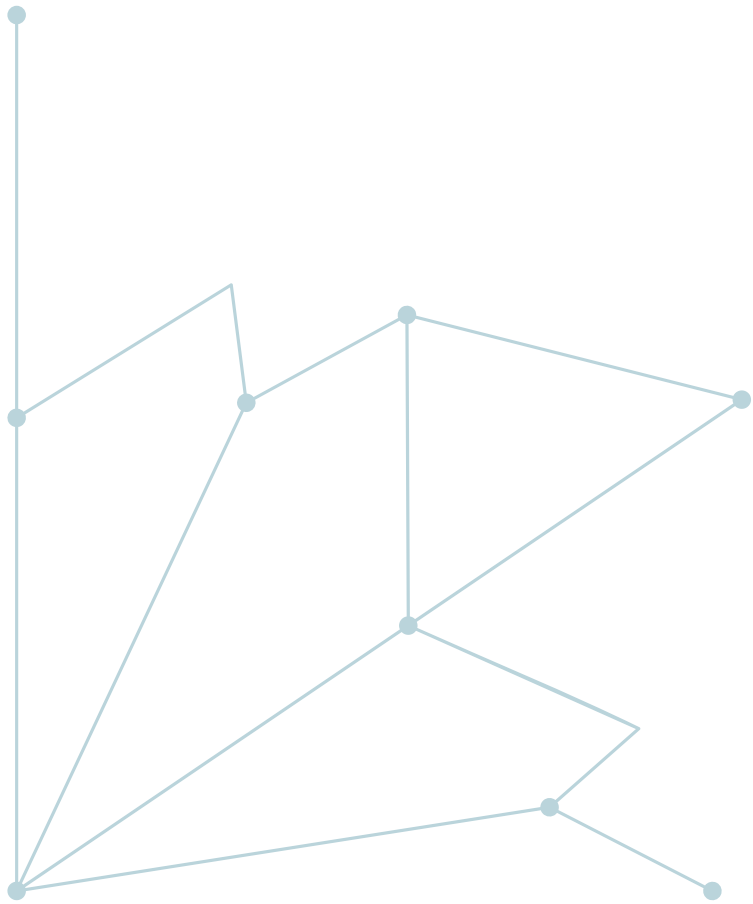
GA – Best Loss 2.5195 – Finished in 2632s ~ 44min

RL – Best Loss 2.5206 – Finished in 580s ~ 10min



Key Insights

- Efficiency of the “Micro-training”:
 - Both algorithms find a good solution under 100 evaluations
- Scalability for industry problems
 - Complexity of the non-convex search space
 - Scalability by DQN-parallelism



Conclusion

1. Survey

- From Static to Dynamic (PBT)
- Hybrid Efficiency (BOHB)
- LLMs Optimizing LLMs

2. Implementation

- Efficiency vs. Performance

3. Perspectives

- Transfer Learning for HPO
 - NanoGPT → Larger models
- Parameter-Efficient Fine-Tuning (PEFT)
 - Optimize Lora Ranks and alphas

